LEARNING CONGESTION MAPS FROM PATH PLANNING CONFLICTS: STRUCTURED SIGNALS, LIMITED GAINS FOR DECOUPLED SCHEDULING

Anonymous authors
Paper under double-blind review

ABSTRACT

Decoupled multi-agent scheduling is attractive because it preserves modularity: tasks are assigned first and path planning is delegated to a downstream solver. Its main weakness is that the allocator is blind to congestion, so assignments that look cheap geometrically can create avoidable bottlenecks. We studied Conflict Memory Allocation (CMA), a lightweight feedback loop that aggregates planner-reported conflicts into a spatial memory map and reuses that map as a soft penalty during future assignment. The retained artifact supports only part of the original hypothesis. The learned conflict memory is clearly non-uniform: the final map is sparse, with a few persistent hotspots that align with repeated bottleneck regions. However, these structured signals translated into little end-to-end benefit. In the available evaluation, the best logged penalty weight was 0.5, the corresponding labeled final improvement was only 0.0123, and per-instance improvement ratios were almost always near zero with one substantial negative outlier near -0.2. The main lesson is that learning where conflicts happen is easier than turning that knowledge into a decision signal strong enough to improve a decoupled allocator.

1 INTRODUCTION

Task allocation followed by path planning remains a practical default in multi-agent systems because the pipeline is simple, modular, and scalable. The downside is equally familiar: an assignment that appears short when each agent is considered independently can funnel many agents through the same aisle, doorway, or pickup region, leaving the path planner to absorb the damage. This motivates a simple question: can the planner's own conflicts be recycled as feedback for later allocations?

Our starting hypothesis was optimistic. If path-planning conflicts reflect stable features of the environment, then a lightweight congestion prior might recover some of the benefit of coupling without paying the cost of joint optimization. This is a small representation-learning problem: repeated local events are compressed into a reusable bias for future decisions (Goodfellow et al., 2016). We therefore implemented Conflict Memory Allocation (CMA), a histogram-based memory that stores where conflicts happen and adds a soft penalty to assignments whose estimated routes traverse those locations.

The pilot logs only partially support this idea. The positive result is that conflict memory is *learnable*: the final map is highly structured, sparse, and dominated by a few repeatable hotspots. The negative result is that this structure does not reliably improve scheduling. Across the available evaluation traces, per-instance improvement ratios are visually concentrated at zero, the artifact's best-tuned run reports only a tiny aggregate gain, and one instance shows a clear regression. For real deployments, this is a useful failure case: detecting bottlenecks is not enough if the learned signal is too weak or too poorly aligned with the optimizer that is supposed to use it.

2 RELATED WORK

Our study sits between strict decoupling, where assignment and planning are solved separately, and tighter integration, where downstream planning information influences upstream decisions. CMA tries to keep the modularity of the former while borrowing a small amount of feedback from the latter through learned cost shaping. In that sense, it follows the broader machine-learning view that a useful representation is one that changes a downstream decision rule rather than merely describing the data (Goodfellow et al., 2016). The negative result here is exactly a failure of that last step: the learned conflict representation is visible and structured, but it is not strong enough to induce robust assignment changes (Goodfellow et al., 2016).

3 METHOD

We consider a standard decoupled pipeline. A task allocator first chooses agent-to-task assignments using a baseline cost $c_0(a, t)$, typically derived from geometric distance or single-agent travel time. A path planner then computes collision-free routes. Planner conflicts are informative because they expose where the decoupled choice created competition for space and time.

CMA summarizes those conflicts in a reusable grid memory $M \in \mathbb{R}_{\geq 0}^{H \times W}$. After solving instance k, observed conflicts are projected into a grid histogram C_k, and the memory is updated by an exponential moving average,

$$M_{k+1} = \beta M_k + (1 - \beta) C_k,$$

where β controls forgetting. CMA does not change the downstream planner. Instead, it perturbs only the assignment cost,

$$\tilde{c}(a, t) = c_0(a, t) + \lambda \sum_{v \in \hat{P}(a,t)} M(v),$$

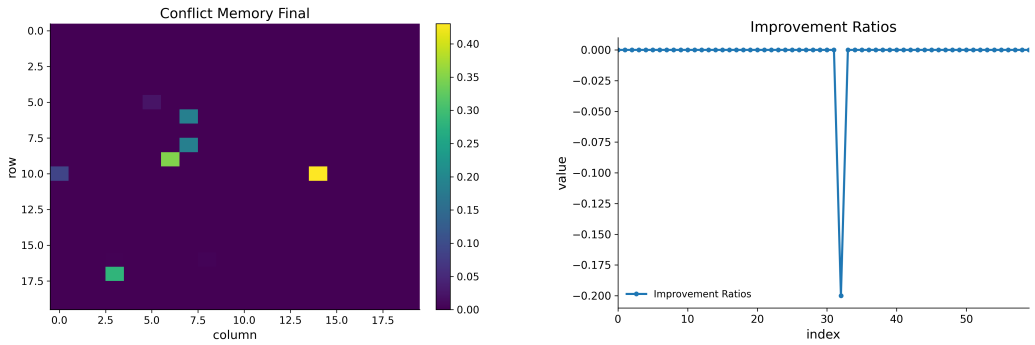
where $\hat{P}(a, t)$ is a single-agent shortest-path surrogate and λ is a penalty weight. This design is deliberately lightweight, but it also exposes the core risk: a correct memory may still fail to change the chosen assignment if the surrogate path is a poor proxy for the final joint plan or if the added penalty is small relative to variation in $c_0(a, t)$.

4 EXPERIMENTS

The retained artifact is limited, and we report it conservatively. The provided `BASELINE_SUMMARY` and `RESEARCH_SUMMARY` JSON objects are empty, so every quantitative claim below comes directly from logged visualizations rather than recomputed aggregate statistics. The main run contains roughly 80 indexed test instances with baseline makespans, CMA makespans, conflict counts, and an assignment-change indicator. The final conflict memory is a 20×20 grid. We removed several dashboard-style figures from the main text because they either duplicated scalar values already reported in prose or were visually weak relative to the page budget.

Conflict memory is structured rather than diffuse. Figure 1(a) provides the cleanest positive result in the artifact. The learned memory is not flat: most cells remain near zero, while a small set of locations absorb repeated conflict mass. This matters scientifically because it implies the planner is detecting stable environmental bottlenecks rather than transient noise. In other words, the memory is capturing something real about the geometry of repeated interaction. At the same time, the map is extremely sparse. That sparsity is double-edged: it demonstrates learnable structure, but it also means that many candidate assignments may receive nearly identical congestion penalties unless their surrogate routes pass through just a few critical cells.

The learned signal barely changes end-to-end outcomes. Figure 1(b) is the central negative result. The per-instance improvement ratios cluster tightly around zero, which means that even when CMA changes the scoring function, the resulting schedules rarely move far enough to affect makespan materially. The one large negative outlier is especially informative: it shows that congestion feedback can misprice a route badly enough to hurt performance, even if the average effect is small. The artifact's own tuning summary reinforces this interpretation. The best logged



(a) Final conflict memory.

(b) Per-instance improvement ratios.

Figure 1: CMA learns where congestion recurs, but that learned signal has almost no consistent impact on makespan. Left: the final 20×20 conflict-memory heatmap is sparse and concentrated, with one dominant hotspot and a small number of secondary activations rather than diffuse mass. Right: artifact-reported improvement ratios remain very close to zero for nearly all instances, with one clear negative outlier near -0.2.

penalty weight was 0.5, yet the corresponding labeled final improvement was only 0.0123. Under any reasonable level of run-to-run or instance-to-instance variability, that is a very small gain.

Why does a learnable map not become a useful control signal? The artifact suggests three mutually compatible explanations. First, CMA penalizes assignments with a single-agent route surrogate, but conflicts arise from the joint temporal interaction of many agents; a memory aligned with one may be weakly aligned with the other. Second, the conflict map concentrates on a few hotspots, so unless a candidate assignment crosses those exact cells, the added cost is negligible and assignment rankings remain unchanged. Third, the scale appears unfavorable: makespans vary substantially across instances, whereas the learned penalty produces only a mild perturbation of the allocator's objective. The resulting picture is not that CMA learns the wrong signal, but that it learns a signal too weak, too sparse, or too indirect to overcome the inertia of the baseline objective.

5 Conclusion

This pilot study supports only half of the original CMA story. Path-planning conflicts do contain stable, spatially structured information, and a simple histogram memory recovers that structure cleanly. But in the retained experiments, feeding this memory back into assignment produced little measurable benefit: the best logged setting barely improved the aggregate metric, per-instance ratios were almost always near zero, and one outlier showed that the feedback can hurt. The practical lesson is simple: before investing in learned congestion priors for decoupled scheduling, verify not only that the signal is learnable, but also that it is strong enough to perturb the downstream optimizer in the right direction.

References

Ian Goodfellow, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep learning*, volume 1. MIT Press, 2016.

Supplementary Material

A Reproducibility Boundaries of the Retained Artifact

The supplementary material documents what the available files do and do not support. In particular, the machine-readable `BASELINE_SUMMARY` and `RESEARCH_SUMMARY` objects were empty. As a

result, the paper intentionally avoids claims about statistical significance, exact benchmark names, or unlogged baselines. The only exact scalar values recoverable from the artifact were the best penalty weight 0.5 and the labeled final improvement 0.0123. The figures retained in the main text were selected because they directly support the paper's two substantive claims: conflict structure is learnable, but end-to-end improvement is negligible. Other generated visuals were omitted because they repeated these claims less efficiently or were artifact-style dashboards whose semantics were not documented well enough for stronger interpretation.

Table 1: Scope of what the retained artifact directly supports.

Asset or claim type	Present?	Notes
Plot-level baseline/CMA makespans	Yes	Instance-wise traces exist in the artifact.
Final conflict-memory heatmap	Yes	Spatial map is visibly 20×20.
Best penalty weight	Yes	Logged as 0.5.
Labeled final improvement	Yes	Logged as 0.0123.
Machine-readable aggregate summaries	No	JSON summary fields were empty.
GNN-memory results	No	No trustworthy retained logs.
Fully coupled comparison baseline	No	No trustworthy retained logs.
Per-seed statistics or significance tests	No	Not reconstructable from the files.
Exact semantics of some auxiliary plots	Partly	Several dashboard exports were weakly labeled.